replacement policies for a set-associative cache

options include:

- (pseudo) random
- LRU ("least recently used")
- approximate LRU

LRU motivated by temporal locality ... blocks recently accessed are likely to be accessed again soon and should be kept in the cache

approximate LRU motivated by the cost of true LRU for large E

access time for a set-associative cache?

when doing a lookup to see if cache contains memory block i, check in set i mod M, similarly as for a direct-mapped cache

now, however, need to check multiple tag fields (specifically, E tag fields)

comparisons with the tag field value that would be used for memory block i can be done in parallel

set-associative cache example

as in previous example, suppose do reads on the following memory addresses: 80, 1040, 948, 1864, 836, and suppose the block size B = 32 bytes

consider cache that can store same total number of memory blocks as the direct-mapped cache in previous example, but now with M = 8 and E = 2

memory block #	cache set	tag field value
2	2 (= 2 mod 8)	0 (= int. part of 2/8)
32	0 (= 32 mod 8)	4 (= int. part of 32/8)
29	5 (= 29 mod 8)	3 (= int. part of 29/8)
58	2 (= 58 mod 8)	7 (= int. part of 58/8)
26	2 (= 26 mod 8)	3 (= int. part of 26/8)

final cache contents

assuming an LRU replacement policy, memory blocks 32, 29, 58, 26 (not memory block 2)

suppose now we read the word with memory address 1888, will we get a cache hit or a cache miss?

design choices for processor caches

must decide:

- number of levels of cache
- size of each cache
- memory block size (often the same for all levels of cache, but sometimes different)
- associativity of each cache and replacement policy if set associative

various tradeoffs to consider ... for example, suppose fix the total number of bytes cache can store, and consider the choice of memory block size ...

Virtual memory addressing

(reference: text Section 5.7)

In machine/assembly language, use the *abstraction* of a memory space … how can we implement this abstraction?

- if knew what main memory locations our program code and data will occupy, when executable is created, our memory space abstraction could correspond directly to the hardware main memory
- commonly, however, when executable is created, the main memory locations the program will occupy when it is loaded into memory and run are unknown!

with virtual memory addressing, we can use any memory space addresses we want when creating an executable (although in practice, will use conventions) … these addresses termed virtual addresses

… at *run time*, these virtual addresses are translated into actual main memory addresses (called "physical addresses"), according to where in main memory our code and data actually reside

benefits of virtual memory addressing

ease of use

- memory space abstraction provides illusion of a large & private memory
- actual location of program in main memory need not be determined until program is run
- program need not be stored contiguously, and can be moved during execution

protection

- *not possible* for a program to *access a part of main memory that does not belong to it*
- crucial mechanism for allowing multiple applications to safely share a machine

facilitates use of disk as backing store

- can run a very large program that doesn't completely fit in main memory
- can run a number of programs concurrently even if they don't all completely fit

overview of virtual memory address translation

divide virtual address space (range of possible memory addresses in machine/assembly language memory space abstraction, in RISC-V RV32I, 0 to $2^{32}-1$) into "virtual pages"

- e.g., with 4 KiB (= 4 x 2^{10} = 4096 byte) pages:

virtual page #	addresses
0	0 - 4095
1	4096 - 8191
2	8192 - 12287
...	...

the portion of an executable that has been assigned the addresses of a virtual page *i* is referred to as virtual page *i* of that executable

divide main memory into "physical pages" of the same size as virtual pages

when a program is loaded into memory, each of its virtual pages can be placed in *any* physical page

operating system maintains a page table for *each* process (program in execution) that records, for each of the virtual pages of the process, which physical page (if any) it has been placed in

a simple page table organization:

- one entry for each virtual page in the virtual address space
- use virtual page number to index into table
- each entry *i* contains some status bits, and (for a valid entry) the number of the physical page into which virtual page *i* has been loaded

e.g.:

status bits (not shown)	physical page number
	3
	8
	5
.....	23 .....

suppose page size is 4 KiB, what *physical address* should *virtual address* 8376 be translated into?

manual address translation

step 1: which virtual page is this?

- VPN given by *integer part* of 8376/4096 (2) (the remainder (184) gives the *offset* of this address within its virtual page)

step 2: which physical page has this virtual page been put in?

- use VPN to index into page table, finding that virtual page 2 has been put in physical page 5

step 3: add offset to starting address of physical page, giving 4096 x 5 + 184 = 20664

hardware address translation

VPN and offset can be found by *extracting fields of the virtual memory address*

- e.g., with 4 KiB = 2^{12} byte pages, rightmost 12 bits give the offset, rest of the bits give the VPN

hardware uses the VPN to index into the page table, and then replaces the VPN with the physical page #

(from text Fig. 4.73)